

Session recap — 2026-08-17

1. Company questions backend redesign

Replaced the old flat, stale `company_questions` table (2200+ rows, no migration history, no import tooling) with a normalized schema sourced from `github.com/liquidslr/leetcode-company-wise-problems`.

New schema (`supabase/migrations/`):

- `lc_companies` — the ~470 LeetCode-tag companies, `name_lower` generated column for case-insensitive uniqueness. Distinct from the pipeline's own `companies` table — matched by free-text name only, no FK.
- `lc_questions` — deduped by LeetCode slug (title, difficulty, leetcode_url).
- `lc_topics` — topic/tag dimension (e.g. "Array", "Hash Table").
- `lc_question_topics` — question ↔ topic many-to-many join.
- `lc_company_questions` — fact table: `company_id`, `question_id`, `time_window` (`30days` | `3months` | `6months` | `6months_plus` | `all_time`), matching the source's 5 CSVs exactly instead of the old 3 buckets, `frequency_score`, `acceptance_rate`.
- RLS: permissive `using (true)` with `check (true)` for anon/authenticated, matching the app's existing no-auth convention.
- Old `company_questions` table dropped after verification.

Import script (`scripts/import-company-questions.ts`, `npm run import:company-questions`): fetches all 470 companies × 5 CSVs (2350 files) from the source repo's raw GitHub URLs, parses with `papaparse`, upserts dimensions then facts, fully idempotent (natural-key conflict targets on every upsert). Needs `SUPABASE_SERVICE_ROLE_KEY` in `.env.local` (never `VITE_`-prefixed). Result: 470 companies, 3392 unique questions, 173 topics, 37714 fact rows.

Frontend rewiring: `src/lib/companyQuestions.ts` rewritten to a nested Supabase select across the new tables; `CompanyQuestion` gained a `topics: string[]` field; `CompanyQuestions.tsx` updated for the 5 time-window buckets and a topics chip line per row.

Bug found and fixed during verification: the frontend query had no pagination, so Supabase's 1000-row response cap silently truncated results — popular companies (Google's `all_time` bucket alone has 2011 rows) showed zero questions for the default filter. Fixed with a paginating fetch loop (`fetchAllCompanyQuestionRows`, `.range()` loop ordered by `id`). Verified live against Google: 2011 rows for `all_time`, 183 for `30 days`, matching the DB exactly.

2. Details / Company questions equal-width layout fix

The two side-by-side cards below the pipeline stepper (`Details` and `Company questions`) were rendering at very unequal widths despite `grid-template-columns: 1fr 1fr`.

Root cause: `.company-questions__topics` uses `white-space: nowrap`, and nothing in the ancestor chain (`.card`, `.company-questions`, the list, the row) had `min-width: 0` to cap it — the long unbreakable topic text forced a large min-content width that blew out the grid track past 50%.

Fix: added `min-width: 0` down the whole chain (`.card`, `.card > *:last-child`, `.company-questions`, `.company-questions__list`, `.company-questions__row`) and changed `.company-detail__row`'s grid columns to `minmax(0, 1fr) minmax(0, 1fr)`. Verified: both cards exactly 533px wide at desktop, still stack correctly below the existing 980px breakpoint.

3. Topic profile radar chart

New feature: a radar/spider chart + matching bar list showing which concepts a company's questions actually test, weighted by difficulty and frequency. Lives directly below the pipeline status stepper, only rendered when a company has LeetCode question data.

Math (`src/lib/topicProfile.ts`):

```
raw(t) = Σ ( freq(p)/100 × difficultyWeight(p) )   for every problem p tagged with topic t
score(t) = raw(t) / max(raw) × 100
```

`difficultyWeight` : easy=1, medium=2, hard=3. Scoped per selected time window; a problem tagged with multiple topics contributes to each independently. Pure client-side computation — no new backend query, reuses the already-fetched `CompanyQuestion[]` from `useCompanyQuestions` .

Components:

- `RadarChart.tsx` — generic hand-rolled inline SVG radar (no charting library, consistent with the app's zero-dependency CSS/JS convention). Concentric grid rings, spoke lines, data polygon, vertex dots, axis labels — all via theme CSS custom properties (`var(--accent)` , `var(--border)` , `var(--text-dim)`) so it themes correctly in light/dark.
- `TopicProfile.tsx` — owns its own time-window `<select>` state (same convention as `CompanyQuestions.tsx`), computes scores via `computeTopicScores` , renders the radar + a scrollable bar list.
- Wired into `CompanyDetail.tsx` directly after `<PipelineTracker />` , gated on `questions.length > 0` .
- `TIME_WINDOWS` hoisted from `CompanyQuestions.tsx` into `companyQuestions.ts` as a shared constant so both components use the same 5 options.

Iterative fixes during verification (real data exposed issues a mockup with 8 topics didn't anticipate):

1. **Radar cap**: showing *all* topics (Google has 168 distinct topics) made the radar unreadable label soup. Capped the radar to the top 16 by score; the bar list still shows every topic, scrollable.
2. **Label clipping / chart too small**: the radar's container box was sized too tightly relative to its label bleed, clipping long labels (e.g. "Breadth-First Search") off the left edge. Fixed by making the radius a fixed proportion of the SVG's viewBox size (not `size/2 - fixed px`) so there's always genuine reserved margin for labels, scaling correctly at any rendered size.
3. **Overflow at medium viewport widths**: `.topic-profile__bars` had a fixed `min-width: 260px` that couldn't shrink, causing real page overflow when the radar + bars didn't both fit. Fixed with `flex-wrap` on the container so the bar list naturally drops below the radar whenever there isn't room, instead of forcing an overflow.
4. Sized down per user feedback (chart was a bit too large; bar list max-height dialed back to sit closer to the chart's height) — current values: radar `width: clamp(300px, 36vw, 460px)` , bars `max-height: clamp(280px, 30vw, 400px)` .

Status: implemented and type-checks clean; last sizing tweak (bar list height) was applied and mid-verification in the browser when this recap was written — worth a final visual confirmation pass in the running app.